CLASSIFIED FLIGHT MANUAL // REV 2026.11

# CHASMA: SURVIVE THE HOLLOWSTAR

A Complete Visual Architecture, Newtonian Physics, Neural AI & Online Netcode Infographic of the Codebase



| 5,544 | 31 PY + 8 TS | 25.6 KB | < 50 us | 0 Files | 60 Hz |
|---|---|---|---|---|---|
| TOTAL CODEBASE LOC | DESKTOP & WEB MODULES | ONNX BRAIN (L1 CACHE) | CPU AI INFERENCE | EXTERNAL AUDIO/SPRITES | DESKTOP & WS NET TICK |

## 1. 1988 AMIGA HERITAGE & CORE PREMISE

EBONSTAR HOMAGE

Inspired by the classic 1988 Commodore Amiga title Ebonstar (by Robert 'Wolf' McNally / The Dreamers Guild), Chasma reinvents 4-player arena dogfighting across both couch local play and instant URL-shareable browser rooms.

### THE ZERO-DAMAGE KINETIC KNOCKBACK PARADIGM

Blasters deal ZERO HP damage. Instead, every shot imparts heavy kinetic knockback (v = 5.0), and ship collisions transfer elastic momentum (e = 0.8). Victory requires banking shots off walls and shoving opponents across the event horizon of the wandering central singularity: The Hollowstar!

### FUN CODEBASE FACTS & ENGINEERING HIGHLIGHTS:

* Shared Pure-Python Kernel: chasma/core/simulation.py (630 LOC) runs identically inside the Desktop OpenGL window and the 60Hz FastAPI WebSocket GameRoom!
* 100% Procedural Everything: Zero .png or .wav files! Every hull, flame, warped grid vertex, and all 8 synth sounds are synthesized in RAM (pyglet + Web Audio).
* Hot-Swap ONNX AI Backfill: Empty slots (0-4) are seamlessly piloted by a 25.6 KB neural network; online players hot-swap in/out mid-match with zero downtime.

## 2. DESKTOP, AI & ONLINE TECH STACK

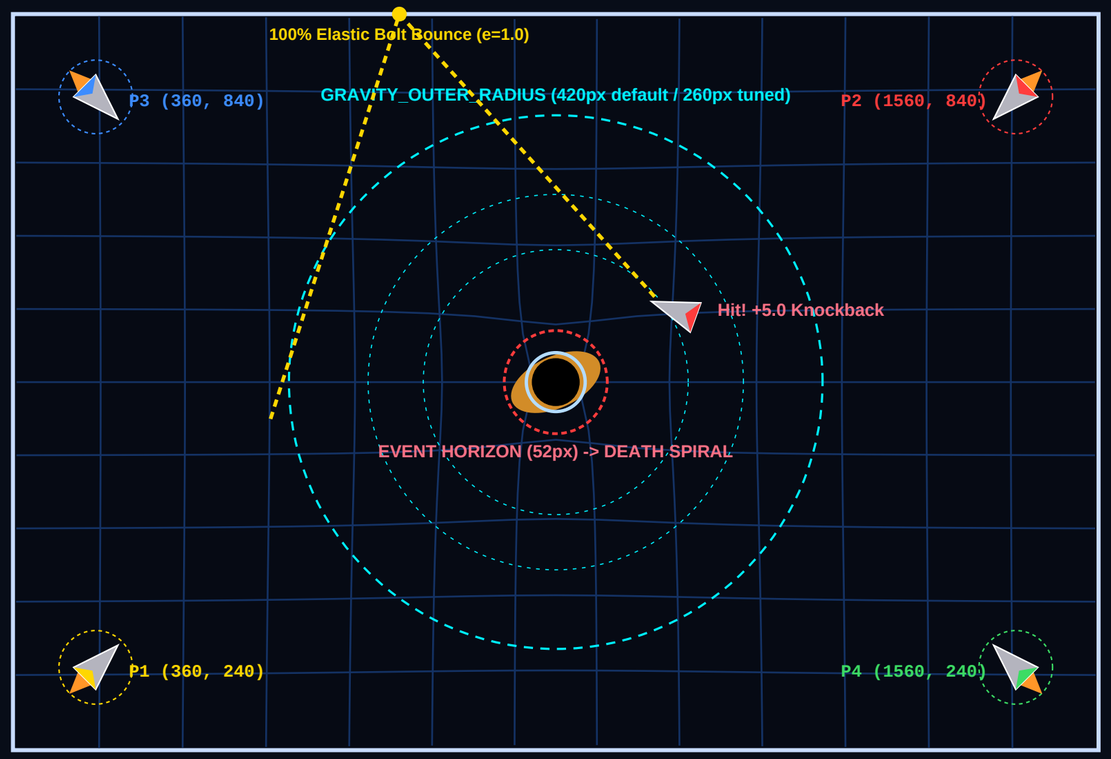
PYTHON + VUE 3 / TS

| LAYER | PACKAGE / API | ARCHITECTURAL ROLE |
|---|---|---|
| **Desktop 2D & GL** | `arcade >= 3.0.0` | OpenGL batch shapes, 60Hz loop, scene Views |
| **Online Netcode** | `fastapi + uvicorn` | 60Hz authoritative rooms, WS & REST endpoints |
| **Browser Client** | `Vue 3 + Canvas2D` | Client prediction, 45ms lerp & 1920x1080 render |
| **Dual Synth Audio** | `pyglet / WebAudio` | Zero-asset procedural oscillators & ADSR |
| **Neural Runtime** | `onnxruntime >= 1.18` | C++ sub-50us CPU policy inference (Opset 18) |
| **RL Environment** | `gymnasium >= 1.0` | Headless 50,000 FPS training arena simulation |
| **Policy & Export** | `sb3 + torch >= 2.2` | PPO Actor-Critic optimization & ONNX export |

## 3. TACTICAL ARENA BLUEPRINT (1920 x 1080 LETTERBOXED TO 1400 x 800)

64px WARPED SPACETIME

WINDOW: 1920 x 1080 FULL HD | ARENA_MARGIN_X = 260px | ARENA_MARGIN_Y = 140px




ARENA BOUNDS: X in [260, 1660] (1400px) | Y in [140, 940] (800px) | Ship Wall Bounce = 0.5x

# SHARED SIMULATION KERNEL & MULTI-PACKAGE TOPOLOGY

How simulation.py unifies Desktop OpenGL (Arcade) and Online 60Hz Multiplayer (FastAPI + Vue 3)



## 1. PACKAGE TOPOLOGY & MODULE DEPENDENCY GRAPH (31 PY + 8 TS/VUE)

5,544 TOTAL LOC

**DESKTOP: main.py (12 LOC) + game.py (48)**
ChasmaGame(arcade.Window) -> TitleView -> GameView

**ONLINE: online/server/app.py (108 LOC)**
FastAPI + /ws/{room} -> RoomManager -> GameRoom

### chasma.core (737 LOC)

**simulation.py (630 LOC) [NEW]**
Zero-GL ChasmaSimulation.step()
ShipState, HollowstarState,
ProjectileState, ArenaBounds

**game.py (48 LOC)**
Window, controller init & F1 hook

**arena.py (59 LOC)**
Arena(ArenaBounds) OpenGL grid
12-seg warped spacetime lines

### chasma.views (322 LOC)

title_view.py (60 LOC)

**game_view.py (128 LOC)**
Wraps ChasmaSimulation;
draws entities & plays events

pause_view.py (71 LOC)

game_over_view.py (63 LOC)
Modal pause & match winner UI

### chasma.ai (769 LOC)

**ai_controller.py (272 LOC)**
14-float obs + ORT + heuristic
Shared by Desktop & GameRoom

**training_env.py (351 LOC)**
ChasmaArenaEnv & HeadlessShip

**train.py (137 LOC)**
PPO -> chasma_pilot.onnx (25KB)

### chasma.entities (203 LOC)

Subclass simulation.py + OpenGL draw

**player.py (83 LOC)**
Player(ShipState): ShapeElementList
dual-triangle mesh & thrust flame

**hollowstar.py (54 LOC)**
Hollowstar(HollowstarState): 4-tier
accretion disc, glow & singularity

**projectile.py (36 LOC)**
Projectile(ProjectileState): 20px bolt

base_entity.py (30 LOC)
Legacy OpenGL entity base class

### chasma.systems (368 LOC)

**input_manager.py (99 LOC)**
1-4 gamepads + keyboard fallback
emits simulation.InputState

**sound.py (114 LOC)**
8 procedural synth waveforms
in RAM + headless CI fallback

**debug_tuner.py (155 LOC)**
F1 live overlay, gravity contour
rings & tuning.overrides.json

### online/ & tests/ (2,811 LOC)

**online/server/ (529 LOC)**
app.py, game_room.py (252),
protocol.py (99), room_manager (69)

**online/scripts/ (139 LOC)**
sync_online.py -> generated_config.ts

**online/web/src/ (1,911 LOC)**
App.vue (719), renderer.ts (363),
prediction.ts (360), sound.ts (163)

**tests/ (232 LOC)**
test_simulation.py (153),
test_server.py (79)

### chasma.config (221 LOC — Centralized Constants, Headless Color Fallback & Live Tuning)

settings.py (84 LOC): Screen (1920x1080), Arena margins, SHIP_SCALE=1.2, Gravity & Blaster constants

colors.py (17 LOC): ChasmaColors palette with headless RGB tuple fallback when arcade is absent

controls.py (5 LOC): STICK_DEADZONE = 0.20, TRIGGER_DEADZONE = 0.05

tuning.py (115 LOC): Overlays settings.py with tuning.overrides.json; hot-mutates globals in-place!

sync_online.py (139): Exports settings + tuning + colors -> online/web/src/config/generated_config.ts

## 2. AUTHORITATIVE 60 HZ PIPELINE (ChasmaSimulation.step)

16.67 ms BUDGET

**1. Inverse-Square Gravity Field**
Pull alive, non-spiraling ships inside R_outer toward Hollowstar

**2. Respawn Timer & Input / AI Dispatch**
Tick respawn_timer; poll human InputState or AIController (<50us)

**3. Ship Physics & Wall Dampening**
Integrate rotation, thrust, 0.99 drag; arena.bounce(dampening=0.5)

**4. Rising-Edge Blaster Fire**
player.try_fire() spawns ProjectileState at nose (22*scale); emits 'fire'

**5. Elastic Ship-to-Ship Collisions**
Separate overlap + restitution impulse (e=0.8, boost=1.2); emits 'hit'

**6. Spiral of Death Sequence**
d < 52px triggers begin_spiral() ('suck'); 50f vortex -> die() ('death')

**7. Projectile & Hollowstar Motion**
Advance bolts (1.0x bounce) + check_hit (+5.0 knockback); wander core

**8. Match Victory & Event Emission**
Return SimulationStepResult(tick, events, is_game_over, winner)

## 3. DESKTOP VIEWS, ONLINE ROOMS & SHIP LIFECYCLE

DUAL RUNTIME

**TitleView**
Press ENTER/START

**GameView**
ChasmaSimulation

**PauseView**
ESC / START Modal

**GameOverView**
<= 1 Ship Alive

**SHIP & ONLINE ROOM LIFECYCLE (3 LIVES):**
Ship: Active -> Crosses 52px -> Spiraling (50f) -> Dead (90f) -> Respawn
Online Room: playing (60Hz broadcast) -> game_over (5.0s cd) -> reset_match()

ASTROPHYSICS ENGINE // CLASSICAL MECHANICS

# NEWTONIAN INERTIA, SPACETIME WARPING & ELASTIC MOMENTUM

Exact mathematical equations in chasma/core/simulation.py powering singularity gravity, grid distortion, and knockback

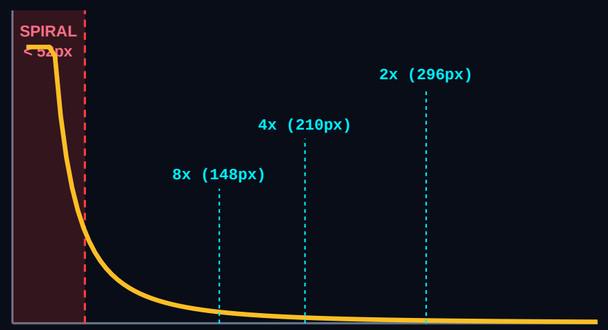


## 1. SINGULARITY & DEATH SPIRAL

INVERSE-SQUARE

HollowstarState drifts at 0.9 px/frame with Brownian heading noise (-0.03 to +0.03 rad/frame) plus a gentle centering spring force (* 0.005).

```
clamped_d = max(dist, GRAVITY_MIN_DIST)
force = GRAVITY_STRENGTH / (clamped_d ** 2)
v += (delta_p / dist) * force
```




Distance from Hollowstar Center (px) -> R_outer = 420px

### THE SPIRAL OF DEATH SEQUENCE (d < 52px):

| | |
|---|---|
| **1. Lock Controls:** | player.spiraling = True; emits 90 Hz 'suck' event. |
| **2. Rapid Spin:** | Ship rotates uncontrollably at SPIRAL_SPIN_SPEED (15 deg/f). |
| **3. Vortex Pull:** | Accelerates inward at 8.0 * GRAVITY_STRENGTH / d^2. |
| **4. Speed Clamp:** | Velocity clamped to MAX_SPIRAL_SPEED = 10.0 (no overshoot). |
| **5. Consumption:** | After SPIRAL_DURATION (50f), ship dies (-1 life, 'death' event). |
| **6. Respawn Delay:** | Waits RESPAWN_DELAY = 90 frames (1.5s) before corner reset. |

## 2. SPACETIME WARP & SHIP MESH

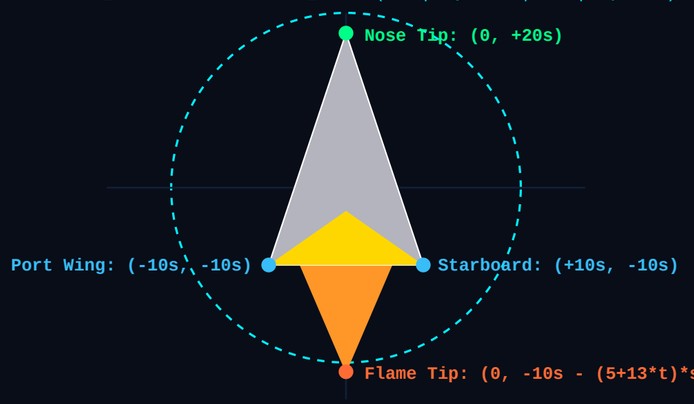
OPENGL + CANVAS2D

Every 64px grid line is split into 12 segments.
Each vertex (px, py) is pulled toward the Hollowstar in both Arena._warp_point() & renderer.ts:

```
warp = GRID_WARP_STRENGTH / (d + FALLOFF)
warp = 1800 / (max(hypot(dx, dy), 1) + 60)
p_warped = p - ((p - h) / d) * warp
```

### PROCEDURAL SHIP POLYGON & COLLISION SPHERE

collision_radius = 14.0 * SHIP_SCALE (16.8px @ 1.2x | 19.6px @ 1.4x)




### COMPASS HEADING & NEWTONIAN DRAG:

In Chasma, 0 deg points North (+Y) & 90 deg points East (+X):

```
rad = math.radians(self.angle)
self.velocity_x += math.sin(rad) * self.thrust
self.velocity_y += math.cos(rad) * self.thrust
self.velocity_x *= DRAG (0.99)  # THRUST_FORCE = 0.15
```

Rising-edge fire latch (_fire_held) prevents holding Fire from auto-spamming bolts; enforces 45-frame cooldown.

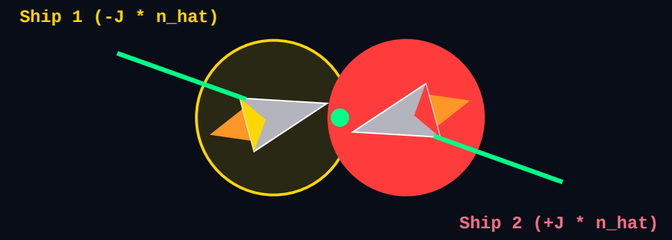
## 3. ELASTIC IMPULSE & KNOCKBACK

MOMENTUM TRANSFER

When two ships collide (dist < r1 + r2), ChasmaSimulation executes a 2-phase rigid-body impulse solver with restitution e = 0.8 and a minimum bump floor:

```
1. Push Apart: p1 -= n*(ov/2); p2 += n*(ov/2)
2. Impulse: j = -(1 + 0.8) * (v_rel . n) * 0.5
3. Total:   J = (j + 1.0) * COLLISION_STRENGTH
```

### ELASTIC SHIP-TO-SHIP COLLISION (e = 0.8)




Minimum bump (1.0) ensures stationary ships still separate!

### BLASTER BOLT KINETICS (ProjectileState):

| | |
|---|---|
| **Bolt Velocity** | BLASTER_SPEED = 8.0 px/frame |
| **Bolt Lifetime** | BLASTER_LIFETIME = 120 frames (2.0s) |
| **Fire Cooldown** | BLASTER_COOLDOWN = 45 frames (0.75s) |
| **Visual Geometry** | 20px line segment, 5px width, owner color |
| **Wall Rebound** | 100% reflection (e=1.0) + atan2 angle update |
| **Hit Radius** | 24px circular check (ignores bolt owner) |
| **Knockback Force** | +5.0 velocity impulse along bolt heading |

# INSIDE THE 25.6 KB NEURAL PILOT: 14 FLOATS -> 3 ACTION HEADS

How AIController encodes ego-centric arena telemetry into L1-cache-resident ONNX inference for Desktop & Online rooms



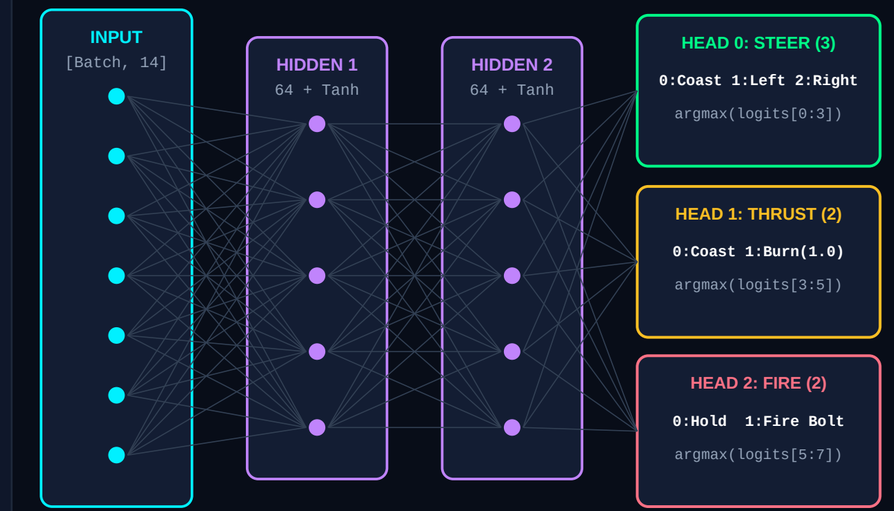
## 1. THE 14-FLOAT EGO-CENTRIC OBSERVATION VECTOR (dtype=float32)

NORMALIZED [-1, 1]

| IDX | FEATURE KEY | EXACT FORMULA | RANGE | TACTICAL PURPOSE |
|---|---|---|---|---|
| [0] | self_vx | clip(bot.vx / 10.0, -1, 1) | [-1, 1] | Own horizontal drift & momentum |
| [1] | self_vy | clip(bot.vy / 10.0, -1, 1) | [-1, 1] | Own vertical drift & momentum |
| [2] | sin_angle | sin(radians(bot.angle)) | [-1, 1] | Continuous X nose unit vector |
| [3] | cos_angle | cos(radians(bot.angle)) | [-1, 1] | Continuous Y nose unit vector |
| [4] | rel_target_x | clip((tx - bx) / 1920, -1, 1) | [-1, 1] | Relative X offset to nearest foe |
| [5] | rel_target_y | clip((ty - by) / 1080, -1, 1) | [-1, 1] | Relative Y offset to nearest foe |
| [6] | target_dist | clip(dist / 2202.9, 0, 1) | [0, 1] | Proximity to target (sweet spot) |
| [7] | rel_target_vx | clip((tvx - bvx) / 10, -1, 1) | [-1, 1] | Relative X closing/fleeing speed |
| [8] | rel_target_vy | clip((tvy - bvy) / 10, -1, 1) | [-1, 1] | Relative Y closing/fleeing speed |
| [9] | heading_align | H . D = sin*Dx + cos*Dy | [-1, 1] | **Dot Product (cos a): +1=DEAD AIM!** |
| [10] | rel_hs_x | clip((hx - bx) / 1920, -1, 1) | [-1, 1] | Relative X offset to Hollowstar |
| [11] | rel_hs_y | clip((hy - by) / 1080, -1, 1) | [-1, 1] | Relative Y offset to Hollowstar |
| [12] | hs_dist | clip(hs_dist / 2202.9, 0, 1) | [0, 1] | Normalized distance to singularity |
| [13] | in_grav_zone | 1.0 if d < R_outer else 0.0 | {0, 1} | Binary emergency gravity alarm |

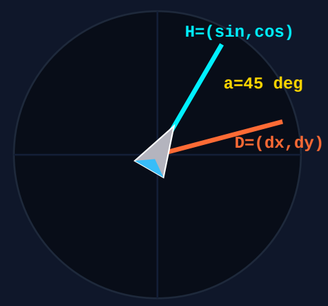
## 2. MLP [64, 64] ACTOR NETWORK & 3 HEADS

OPSET 18 / 25.6 KB




## 3. VECTOR MATH & HEURISTIC FALLBACK

DOT & CROSS PRODUCT




**1. Continuous Unit Circle (sin, cos):**
Eliminates the 359 deg -> 1 deg wrap-around cliff that confuses standard MLP networks.

**2. Dot Product Aiming (H . D = cos a):**
+1.0 = Dead Aim (Fire!) | 0.0 = 90 deg Flank
-1.0 = Target directly behind ship.

**3. 2D Cross Product (H x D = sin a):**
Used by _predict_heuristic() fallback pilot:
cross > 0 -> Turn Right | cross < 0 -> Left!

**FALLBACK PILOT (_predict_heuristic) & ONLINE HOT-SWAP:**
Shared ONNX session pilots empty Desktop & Online GameRoom slots!

REINFORCEMENT LEARNING // GYMNASIUM + PPO

# THE 50,000 FPS HEADLESS FLIGHT ACADEMY & REWARD SHAPING

How ChasmaArenaEnv cures 'Turret Camping' and trains aggressive orbital dogfighters in 250,000 timesteps



## 1. ANTI-CAMPING REWARD FUNCTION (chasma/ai/training_env.py)

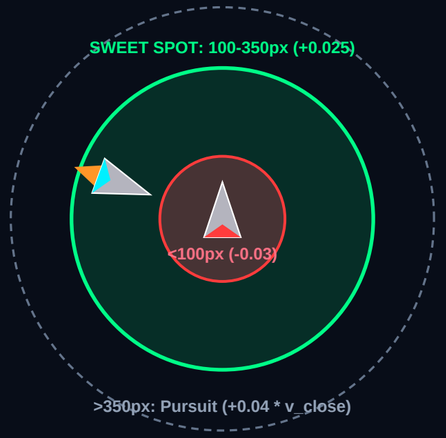
COMMIT 4b758a0

Early RL agents discovered a lazy local optimum: park stationary in a corner, spin like a turret, and farm aiming points. The v2 reward function enforces active dogfighting:

| REWARD / PENALTY TERM | FORMULA / MAGNITUDE | BEHAVIORAL SHAPING EFFECT |
|---|---|---|
| Blaster Hit Impact | +10.0 per hit | Primary offensive jackpot; rewards lead-shot accuracy |
| Singularity Death | -15.0 (Terminal) | Strong survival instinct around Hollowstar (<52px) |
| Gravity Zone Tax | -0.02 * (1 - d/R_out) | Continuous gradient pushing bot out of vortex ring |
| Moving Aim Bonus | +0.04 * max(0, cos a) | ONLY granted if speed >= 1.2! Zero reward if parked! |
| Camping Penalty | -0.03 * (1.2 - speed) | Directly punishes hovering or stationary turret spinning |
| Cruise Speed Bonus | +0.02 (1.8 <= v <= 5.2) | Rewards smooth, controllable orbital strafing speed |
| Sweet-Spot Range | +0.025 (100 <= d <= 350) | Maintains ideal dogfight ring around opponent |
| Anti-Ramming Tax | -0.03 (dist < 100px) | Prevents reckless kamikaze collisions into enemy |
| Pursuit Closing | +0.04 * (closing_v / 6) | When far (d > 350px), rewards closing the gap |
| Throttle Discipline | -0.02 (v>6 or cos<-0.3) | Penalizes thrusting while overspeeding or facing away |

## 2. TACTICAL ENGAGEMENT SWEET-SPOT & VELOCITY GATES

REWARD TOPOLOGY




SHIP SPEED REWARD GATES:

**v < 1.2 px/f (Stagnant / Parked)**
`Aim Bonus = 0! Penalty: -0.03*(1.2-v)`

**v >= 1.2 px/f (Active Motion)**
`Unlocks +0.04 * cos(a) Aim Bonus!`

**1.8 <= v <= 5.2 px/f (Cruise)**
`Optimal dogfight speed (+0.02/frame)`

**v > 6.0 px/f + Burning Thrust**
`Overspeed reckless penalty (-0.02)`

**cos(a) < -0.3 + Burning Thrust**
`Wrong-way thrust penalty (-0.02)`

## 3. PPO HYPERPARAMETERS & ONNX EXPORT (chasma/ai/train.py)

250,000 TIMESTEPS

| | | | |
|---|---|---|---|
| RL Algorithm: | PPO (Proximal Policy Optimization) | Policy Architecture: | MlpPolicy net_arch=[64, 64] (Tanh) |
| Learning Rate: | 3e-4 (Adam optimizer) | Rollout Buffer: | n_steps = 2048 \| batch_size = 64 |
| PPO Epochs & Clip: | n_epochs = 10 \| clip_range = 0.20 | Discount & GAE: | gamma = 0.99 \| gae_lambda = 0.95 |
| Entropy Coef: | ent_coef = 0.01 (encourages exploration) | Episode Horizon: | 1,800 ticks (30 seconds @ 60 FPS) |

```
class OnnxablePolicy(th.nn.Module):  # Strips Critic head & exports pure Actor
  features = self.policy.extract_features(obs, self.policy.pi_features_extractor)
  logits = self.policy.action_net(self.policy.mlp_extractor.forward_actor(features))
  return th.stack([argmax(logits[:,0:3]), argmax(logits[:,3:5]), argmax(logits[:,5:7])], dim=1)
```

# DUAL PROCEDURAL SYNTHESIZERS, F1 LIVE TUNER & CONFIG SYNC

Zero-asset synthesis (sound.py + sound.ts), F1 live tuning (debug_tuner.py), and Desktop-to-Online config sync



## 1. THE 8 PROCEDURAL SYNTHESIZER PATCHES (sound.py & sound.ts)

0 WAV FILES

Synthesizes 8 retro sound effects in RAM via pyglet.media.synthesis (Desktop)
and Web Audio API OscillatorNode / AudioBuffer white noise (Browser):




| PATCH KEY | WAVEFORM | FREQ | DUR | ENVELOPE PROFILE | GAME EVENT |
|---|---|---|---|---|---|
| "fire" | Square | 880 Hz | 0.08s | LinearDecay(peak=0.30) | Blaster bolt fired |
| "hit" | Square | 220 Hz | 0.12s | LinearDecay(peak=0.40) | Blaster or ship impact |
| "death" | WhiteNoise | Noise | 0.40s | LinearDecay(peak=0.35) | Ship consumed in core |
| "bounce" | Triangle | 600 Hz | 0.03s | LinearDecay(peak=0.15) | Ship arena wall bounce |
| "menu_select" | Sine | 1200 Hz | 0.10s | ADSR(0.01,0.05,0.04,0.3) | Menu confirmation |
| "game_over" | Sawtooth | 110 Hz | 0.80s | LinearDecay(peak=0.30) | Match defeat screen |
| "win" | Sine | 880 Hz | 0.30s | ADSR(0.02,0.10,0.15,0.3) | Victory celebration |
| "suck" | Sine | 90 Hz | 0.60s | ADSR(0.05,0.20,0.30,0.25) | Entering death spiral |

**HEADLESS CI FALLBACK & WEB AUDIO PORT (WebSoundManager):**

Desktop SoundManager caches StaticSource() in RAM and silently disables on headless CI.
Online WebSoundManager (online/web/src/game/sound.ts) replicates all 8 patches in browser
using Web Audio API OscillatorNode + GainNode envelopes triggered by server 'ev' packets!

## 2. F1 LIVE TUNER & ONLINE SYNC (sync_online.py)

12 LIVE PARAMS

F1 opens DebugTuner; sync_online.py exports overrides to generated_config.ts & WS welcome:

| PARAMETER KEY | DEFAULT | ACTIVE JSON OVERRIDE | STEP (-/=) | BIG (_/+) |
|---|---|---|---|---|
| SHIP_SCALE | 1.2 | 1.4 [OVERRIDDEN *] | 0.1 | 0.5 |
| SHIP_COLLISION_STRENGTH | 1.2 | 1.2 (Default) | 0.1 | 0.5 |
| GRAVITY_STRENGTH | 3200.0 | 2600.0 [OVERRIDDEN *] | 100 | 500 |
| GRAVITY_OUTER_RADIUS | 420 | 260 [OVERRIDDEN *] | 10 | 40 |
| GRAVITY_INNER_RADIUS | 52 | 52 (Default) | 2 | 10 |
| GRAVITY_MIN_DIST | 30 | 30 (Default) | 1 | 5 |
| SPIRAL_DURATION | 50 | 30 [OVERRIDDEN *] | 2 | 10 |
| SPIRAL_FORCE_MULTIPLIER | 8.0 | 7.4 [OVERRIDDEN *] | 0.2 | 1.0 |
| SPIRAL_SPIN_SPEED | 15 | 14 [OVERRIDDEN *] | 0.5 | 2.0 |
| MAX_SPIRAL_SPEED | 10.0 | 10.0 (Default) | 0.2 | 1.0 |
| GRID_WARP_STRENGTH | 1800 | 1800 (Default) | 50 | 200 |
| GRID_WARP_FALLOFF | 60 | 60 (Default) | 2 | 10 |

## 3. OPERATOR CONTROL DECK (Desktop & Browser Gamepad/Keys)

1-4 PLAYERS

**GAMEPAD (P1-P4 / HTML5)**

```
L-Stick X : Rotate (DZ=0.20)
R-Trigger : Analog Thrust
A / B Btn : Fire Blaster Bolt
START Btn : Edge-Triggered Pause
```

**KEYBOARD (DESKTOP & WEB)**

```
Arrows/AD : Rotate (-1 / +1)
Up / W    : Full Thrust (1.0)
SPACEBAR  : Fire Blaster Bolt
M / F1    : Mute / Debug HUD
```

**F1 TUNER HOTKEYS**

```
F1        : Toggle Overlay & Rings
[ / ]     : Cycle 12 Parameters
-/= & _/+ : Small / Big Adjust
F2/F3/F4  : Reset / Dump / Wipe
```

MULTIPLAYER NETCODE // AUTHORITATIVE SERVER & WEB CLIENT

# CHASMA ONLINE: 60 HZ WEBSOCKETS, CLIENT PREDICTION & DOCKER

How FastAPI GameRooms, compact tuple snapshots, 0ms local prediction, and 45ms interpolation power URL-shareable play



## 1. AUTHORITATIVE SERVER (online/server/)

FASTAPI + UVICORN

online/server/ hosts up to 50 concurrent 4-player arenas in a single async Python process (--workers 1) sharing one L1-cached ONNX AIController across all active rooms:

### app.py (108 LOC) — HTTP + WebSocket Entrypoint

```
GET  /api/health       -> active_rooms & onnx_loaded status
GET  /api/config       -> runtime settings + tuning overrides
POST /api/rooms        -> allocates fresh 4-char room code
GET  /api/rooms/{code} -> room existence, human_count, phase
WS   /ws/{code}?name=  -> upgrades & binds to GameRoom
GET  /{full_path}      -> serves compiled Vue 3 SPA dist/
```

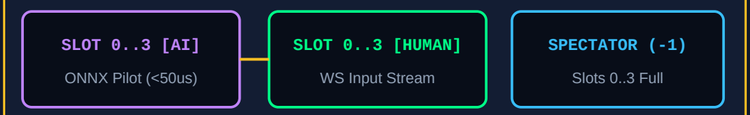
### room_manager.py (69 LOC) — Room Code Coordinator

```
Alphabet: ABCDEFGHJKLMNPQRSTUVWXYZ23456789 (no 0/O/1/I)
MAX_ACTIVE_ROOMS = 50 (evicts oldest empty room if full)
EMPTY_ROOM_GRACE_SEC = 30.0s auto-stop & garbage collection
Shares singleton AIController() across all GameRooms
```

### game_room.py (252 LOC) — Drift-Free 60Hz Loop & Hot-Swap

```
TICK_RATE = 60 (16.67ms) drift-free perf_counter() loop
Slots [0..3]: lowest free slot assigned on WS connect;
  5th+ connections join self.spectators (slot = -1)
Hot-Swap In:  ship.is_ai = False; revives if lives <= 0!
Hot-Swap Out: disconnect -> ship.is_ai = True instant AI!
Captures rising-edge pending_fire between 60Hz ticks
GAME_OVER_DELAY_SEC = 5.0s countdown -> auto reset_match()
```

### DYNAMIC SLOT HOT-SWAP & SPECTATOR OVERFLOW:




```
* Connect:    Assigns first None in slots[0..3]; revives ship
              to 3 lives at corner spawn if AI died earlier.
* Disconnect: Sets slots[i]=None & ship.is_ai=True on the
              exact same frame — match never pauses or drops!
* Commands:   set_name (<=12 chars), reset_match, ping/pong.
```

## 2. 60 HZ WIRE PROTOCOL (protocol.py)

COMPACT JSON TUPLES

Minimizes WebSocket bandwidth at 60 ticks/sec by packing entities into flat positional arrays with rounded floats:

### A. Handshake Packet (Server -> Client on Join)

```
{"t":"welcome", "room":"K7M9", "slot":0,
 "config":{...}, "tuning":{"SHIP_SCALE":1.4,...}}
```

Streams live desktop F1 tuning overrides to browser on connect.

### B. Client Input Packet (Client -> Server @ 60Hz)

```
{"t":"i", "seq":412, "rot":-1.0, "thr":1.0, "fire":1}
```

Server clamps rot in [-1,1], thr in [0,1], latches rising-edge fire, and echoes client's latest seq as 'ack' in next snapshot.

### C. Authoritative World Snapshot (Server -> All @ 60Hz)

```
{"t":"s", "tick":1840, "ack":412, "phase":"playing",
 "cd":0, "winner":null, "hs":[x, y, spin_angle],
 "p":[[slot,x,y,vx,vy,ang,thr,lives,alive,spir,ai,name]],
 "proj":[[id, owner_slot, x, y, angle]],
 "ev":["fire", "hit", "suck", "death", "win"]}
```

12-element player tuple & 5-element bolt tuple (~350 bytes/tick).

## 3. DOCKER & NGINX DEPLOY (online/)

MULTI-STAGE BUILD

### Stage 1: node:22-slim (web-builder)

```
1. Runs python3 online/scripts/sync_online.py
   -> writes online/web/src/config/generated_config.ts
2. Runs npm install && npx vite build -> dist/
```

### Stage 2: python:3.12-slim (chasma-online)

```
Copies chasma/, online/, tuning.overrides.json & dist/
uvicorn online.server.app:app --port 8080 --workers 1
docker-compose.yml binds 0.0.0.0:9999 -> 8080 + healthcheck
```

### NGINX Reverse Proxy (online/nginx/chasma.conf)

```
tcp_nodelay on; proxy_buffering off; proxy_cache off;
Upgrades /ws/{room} (HTTP/1.1) with 3600s read/send timeout
Forwards X-Real-IP / X-Forwarded-For from Cloudflare Tunnel
```

## 4. VUE 3 + CANVAS2D CLIENT (web/src/)

0ms INPUT FEEL

WorldStatePredictor (prediction.ts) combines instant local ship prediction with a 45ms interpolation buffer:

### 1. Local Ship Prediction & Server Reconciliation

```
recordLocalInput() assigns ++seq & appends to pendingInputs[]
stepShipPhysics() runs exact Python thrust, drag (0.99),
  wall bounce (0.5x) & Hollowstar gravity locally in TS!
On server 's' packet: drops inputs where seq <= pkt.ack,
  snaps to authoritative state & replays unacked inputs!
```

### 2. 45ms Remote Entity Interpolation (INTERP_DELAY_MS)

```
Remote ships, projectiles & Hollowstar render at:
  targetTime = performance.now() - 45ms (~2.7 ticks)
Finds bracketing snapshots [s0, s1] and interpolates:
  x, y, vx, vy via lerp(a, b, t)
  angle & spinAngle via shortest-arc lerpAngleDeg()
```

### 3. 1920x1080 Canvas2D Vector Renderer (renderer.ts)

```
Inverts Y axis: cy(worldY) = 1080 - worldY (OpenGL -> Canvas)
Draws 12-segment warped spacetime grid, 4-tier Hollowstar,
  dual-triangle ships, flickering thrust flames & bolts
Draws 'YOU' indicator ring & F1 debug gravity contours + RTT
```

### 4. Vue 3 Lobby, Invite URL & HUD (App.vue — 719 LOC)

```
Quick Play / Create Room / Join 4-Char Code + ?room=ABCD URL
One-click Copy Invite Link, Callsign editor, Mute & F1 toggles
5.0s Game Over overlay with Play Again Now button
```

### DUAL-TIMELINE NETCODE SYNCHRONIZATION:

